On the Reasoning Gaps of Large Language Models: A Formal Characterization

Anonymous Author(s)
Affiliation
Address
email

Abstract

Large language models demonstrate remarkable reasoning abilities yet fail systematically on seemingly simple problems. We show these failures are not random but correspond to *reasoning gaps*—problem classes that fall outside the computational boundary of the model's architecture. Building on transformer expressiveness results that place fixed-depth log-precision transformers within TC^0 (constant-depth threshold circuits), we develop a six-type taxonomy of reasoning gaps, each grounded in a specific complexity-theoretic boundary: *sensitivity* (AC^0), *depth* ($\mathrm{TC}^0/\mathrm{NC}^1$), *serial composition* (linear depth), *algorithmic* (P), *intractability* (NP), and *architectural* (autoregressive constraints). We introduce ReasonGap, a diagnostic benchmark suite of nine tasks designed to isolate each gap type, with testable predictions about when chain-of-thought reasoning, model scaling, and tool augmentation will close or fail to close each gap. Evaluation across twelve models from five families (176,477 instances) validates the taxonomy: chain-of-thought closes depth and serial gaps as predicted by complexity theory, tool use dominates for algorithmic gaps, and architectural gaps show limited or model-dependent response to CoT. The practical implication is direct: given a task's complexity class, one can predict which interventions will help and which will not.

1 Introduction

Large language models (LLMs) achieve striking performance on reasoning benchmarks (Cobbe et al., 2021; Srivastava et al., 2023; Talmor et al., 2019), yet fail on seemingly trivial problems—miscounting objects (Nezhurina et al., 2024), collapsing under irrelevant information (Mirzadeh et al., 2024), or failing to infer "B is A" from "A is B" (Berglund et al., 2024). These failures are not random; they reveal *systematic reasoning gaps* that correlate with structural properties of the underlying problem.

Two disconnected communities have studied this phenomenon. Complexity theorists established that fixed-depth log-precision transformers are bounded by TC^0 (Merrill & Sabharwal, 2022, 2023a), with chain-of-thought (CoT) extending expressiveness to P (Merrill & Sabharwal, 2024; Li et al., 2024). Empiricists catalogued failure patterns in compositionality (Dziri et al., 2023), planning (Kambhampati et al., 2024), and mathematical reasoning (Mirzadeh et al., 2024), without connecting them to a formal framework. We bridge this divide with four contributions:

1. A **formal definition** of *reasoning gap* grounded in complexity theory (Section 3).
2. A **six-type taxonomy** with propositions characterizing existence and closure conditions (Section 3.2).
3. **ReasonGap**, a diagnostic benchmark of nine tasks isolating each gap type (Section 4).
4. **Empirical validation** across twelve models, five families, and 176,477 instances (Section 5).

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Our central thesis: reasoning gaps are *predictable*—given the complexity class of a problem and the computational class of a model, one can determine *a priori* whether a gap exists, its type, and which mitigations (if any) will close it.

2 Background

Circuit complexity. We use standard complexity classes ordered by inclusion: $\mathrm{AC}^0 \subseteq \mathrm{TC}^0 \subseteq \mathrm{NC}^1 \subseteq \mathrm{L} \subseteq \mathrm{NL} \subseteq \mathrm{P} \subseteq \mathrm{NP}$. AC^0 consists of constant-depth polynomial-size circuits with unbounded fan-in; TC^0 adds threshold gates; NC^1 allows $O(\log n)$ depth. All inclusions are believed strict. Intuitively, each boundary marks a qualitative jump: AC^0 captures what a single parallel pass can recognize (pattern matching), TC^0 adds counting and threshold operations (a transformer's forward pass), NC^1 permits $O(\log n)$ sequential steps (tree-structured recursion), and P captures arbitrary polynomial-time algorithms. For a transformer bounded by TC^0, any problem requiring even logarithmic-depth sequential reasoning lies beyond the forward pass—chain-of-thought is the mechanism that extends computation up this hierarchy. Key separations: PARITY $\notin \mathrm{AC}^0$ (Furst et al., 1984; Håstad, 1986); Boolean formula evaluation is NC^1-complete; 3-SAT is NP-complete.

Transformer expressiveness. Fixed-depth log-precision transformers are bounded by uniform TC^0 (Merrill & Sabharwal, 2022, 2023a). CoT extends this: $O(\log n)$ steps reach L, $O(n)$ steps capture regular languages, and $O(n^c)$ steps equal exactly P (Merrill & Sabharwal, 2024; Li et al., 2024). These bounds apply across architectures, including state-space models (Mamba Complexity, 2024). Crucially, expressiveness does not imply learnability: high-sensitivity functions occupy isolated points in parameter space, making them unlearnable via gradient descent (Hahn & Rofin, 2024).

3 Formal Framework

3.1 Definitions

Definition 1 (Reasoning Task)**.** A *reasoning task* is a family $\mathcal{F} = \{f_n : \Sigma^{\leq n} \to \Sigma^*\}_{n\in\mathbb{N}}$ of functions parameterized by input size n, equipped with a decidable correctness criterion $V(x, y) \in \{0, 1\}$ such that $V(x, f_n(x)) = 1$ for all valid inputs x.

Definition 2 (Model Capability Class)**.** Given a model class $\mathcal{M}$ (e.g., depth-d log-precision transformers), the *capability class* $\mathcal{C}(\mathcal{M}) \subseteq 2^{\mathcal{F}}$ is the set of reasoning tasks $\mathcal{F}$ such that $\mathcal{M}$ solves $\mathcal{F}$ with probability $\geq 2/3$ on uniformly random instances of size n, for all sufficiently large n.

Definition 3 (Reasoning Gap)**.** A model class $\mathcal{M}$ has a *reasoning gap* on task $\mathcal{F}$ if:

1. $\mathcal{F} \in \mathrm{P}$ (the task is tractable);
2. $\mathcal{F} \notin \mathcal{C}(\mathcal{M})$ (the task exceeds the model's capability class);
3. The failure is systematic: accuracy either (a) degrades with input size n, or (b) remains bounded away from perfect accuracy by a non-vanishing margin, independently of n and model scaling.

Condition (3a) captures *computational* gaps where difficulty overwhelms the model as instances grow; condition (3b) captures *architectural* gaps where the model achieves partial but never complete accuracy due to structural or training limitations. A gap is *provable* if condition (2) follows from a proven complexity separation, and *empirical* if observed experimentally.

Definition 4 (Gap Closure)**.** An augmentation technique $\mathcal{T}$ (e.g., CoT with budget $b(n)$, tool use, external memory) *closes* a reasoning gap on $\mathcal{F}$ for model $\mathcal{M}$ if $\mathcal{F} \in \mathcal{C}(\mathcal{M} + \mathcal{T})$. A technique *partially closes* a gap if it improves asymptotic accuracy without achieving a constant lower bound.

3.2 Reasoning Gap Taxonomy

We identify six gap types, each defined by the complexity boundary it reflects (Table 1). **Type 1 (Sensitivity, AC^0 boundary):** Problems requiring high Boolean sensitivity, such as exact counting; functions with sensitivity $\Omega(n)$ are not in AC^0 (Linial et al., 1993), and may be unlearnable even

Table 1: Reasoning gap taxonomy with predictions. Each gap type corresponds to a complexity boundary and makes testable predictions about the effectiveness of CoT, scaling, and tool use.

Type	Boundary	Scales with n?	CoT helps?	CoT budget	Tool use?
1. Sensitivity	AC^0	Yes	Partially	$\Omega(n)$	No
2. Depth	$\mathrm{TC}^0/\mathrm{NC}^1$	Yes (nesting)	Yes	$O(\log n)$	No
3. Serial	Linear depth	Yes (steps)	Yes	$O(n)$	Partially
4. Algorithmic	P	Yes	In theory	$O(\mathrm{poly}(n))$	**Yes**
5. Intractability	NP	Yes (phase)	No	—	Only w/ solver
6. Architectural	Structural	No (constant)	Partially	—	Sometimes

when expressible (Hahn & Rofin, 2024). **Type 2 (Depth, $\mathrm{TC}^0/\mathrm{NC}^1$ boundary):** Problems requiring super-constant depth, e.g., Boolean formula evaluation (NC^1-complete); accuracy degrades with nesting depth, and $O(\log n)$ CoT steps suffice (Yehudai et al., 2026). **Type 3 (Serial Composition):** Problems requiring $\Omega(n)$ sequential steps (iterated permutation, state tracking); CoT budget must match the number of serial steps (Merrill & Sabharwal, 2024). **Type 4 (Algorithmic):** P problems requiring non-trivial algorithms; polynomial CoT suffices in theory, but tool use closes the gap far more reliably (Gao et al., 2023). **Type 5 (Intractability):** NP-hard problems where even polynomial CoT reaches only P; accuracy degrades sharply at phase transitions (Hazra et al., 2025). **Type 6 (Architectural):** Failures from autoregressive structure or training objectives (reversal curse (Berglund et al., 2024), negation insensitivity (Negation, 2025)); these persist regardless of scaling or problem size; CoT may partially compensate for inference-time manifestations but cannot resolve the underlying structural limitation.

3.3 Formal Results

We state five propositions; proofs appear in Section A.

Proposition 5 (Sensitivity Gap Existence)**.** *Under standard assumptions, there exist tasks with sensitivity $s(f) = n$ that no constant-depth transformer can solve for all n.*

Proposition 6 (Depth Gap Existence)**.** *If $\mathrm{TC}^0 \neq \mathrm{NC}^1$, there exist NC^1 tasks no constant-depth transformer can solve, but $O(\log n)$ CoT steps suffice.*

Proposition 7 (Serial Gap Characterization)**.** *Iterated composition of k functions on a finite domain requires $\Theta(k)$ CoT steps.*

Proposition 8 (CoT Budget Monotonicity)**.** *For Type 3 gaps on tasks with serial depth k, accuracy under CoT with budget b is non-decreasing in b for $b \leq O(k)$.*

Remark 9. Beyond the sufficient budget $O(k)$, empirical evidence shows accuracy can plateau or decrease—a phenomenon termed "overthinking" (Mirage, 2025). Our budget sensitivity experiments (Section 5.4) confirm monotonicity within the theoretical regime.

Proposition 10 (Intractability Invariance)**.** *For Type 5 gaps, accuracy at the NP-hard phase transition does not improve beyond a constant with any polynomial CoT budget.*

4 The ReasonGap Benchmark Suite

We design nine diagnostic tasks, each targeting a specific gap type with procedurally generated instances to avoid contamination (Table 2). B5 (Graph Reachability) serves as a bridge task: as an NL-complete problem, it sits between depth gaps (Type 2) and algorithmic gaps (Type 4), probing the boundary where additional depth may or may not suffice. Each task is evaluated at five difficulty levels (100 instances each) under four conditions: direct, short CoT, budget CoT (task-calibrated token limits), and tool-augmented (code execution, where applicable). Full task specifications appear in Section B.

5 Experiments

We evaluate the ReasonGap benchmark across twelve models from five families under four reasoning conditions, totaling 176,477 instances.

Table 2: REASONGAP benchmark tasks. Each task targets a specific gap type with a controlled difficulty parameter and a testable prediction.

ID	Task	Gap Type	Complexity	Difficulty	CoT Prediction
B1	Masked Majority	Sensitivity	$\mathrm{TC}^0 \setminus \mathrm{AC}^0$	String length n	Short CoT closes
B2	Nested Boolean Eval	Depth	NC^1-complete	Nesting depth d	$O(\log n)$ CoT closes
B3	Iterated Permutation	Serial	$\Omega(k)$ serial	Compositions k	$O(k)$ CoT required
B4	State Machine Sim	Serial	$\Omega(k)$ serial	Transitions k	$O(k)$ CoT required
B5	Graph Reachability	Depth/Algo	NL-complete	Nodes n	Partial improvement
B6	Longest Incr. Subseq	Algorithmic	P	Sequence length	Tool use dominates
B7	3-SAT	Intractability	NP-complete	Variables n	No improvement
B8	Reversal Inference	Architectural	Structural	Distractor count	No improvement
B9	Negation Sensitivity	Architectural	Structural	Negation depth	Minimal

Table 3: Overall accuracy by task and model (averaged across conditions). Bold: best model per task. Accuracy varies far more across tasks than models, confirming gap type as the primary difficulty determinant. Random baselines: 0.50 for B1, B2, B5, B7, B9 (binary/boolean); 0.20 for B3 (5-way); 0.25 for B4 (4-way); $\sim 1/(d{+}1)$ for B8 (d distractors); $\sim 1/n$ for B6 (numeric).

	Haiku 4.5	*Sonnet 4.6*	*Opus 4.6*	*GPT-4o-m*	*GPT-4o*	*o3*	*Llama 8B*	*Llama 70B*	*Ministral 8B*	*Mistral 24B*	*Qwen 7B*	*Qwen 72B*
B1: Majority	.63	.68	.70	.72	.78	.13	.55	.68	.72	.72	.67	**.81**
B2: Bool Eval	.80	.72	.83	.83	.82	.50	.67	**.84**	.81	.80	.74	.84
B3: Perm Comp	.41	.30	**.81**	.58	.53	.25	.27	.60	.62	.57	.31	.62
B4: State Track	.71	.74	**.79**	.62	.55	.35	.48	.69	.66	.68	.52	.72
B5: Graph Reach	.79	.72	.79	.79	.83	.46	.51	.70	.78	.65	.66	**.83**
B6: LIS	.48	.50	**.62**	.30	.37	–	.21	.24	.24	.22	.28	.35
B7: 3-SAT	.37	.41	.56	.63	**.67**	.12	.32	.34	.42	.51	.56	.61
B8: Reversal	1.00	**1.00**	**1.00**	.94	.99	.74	.97	1.00	.99	.98	.95	.99
B9: Negation	.73	.78	**.79**	.54	.57	.31	.50	.74	.56	.57	.59	.75

5.1 Setup

We evaluate twelve models: Claude Haiku 4.5, Sonnet 4.6, and Opus 4.6 (Anthropic); GPT-4o, GPT-4o-mini, and o3 (OpenAI); Llama 3.1 8B and 70B (Meta); Ministral 8B and Mistral Small 24B (Mistral); Qwen 2.5 7B and 72B (Alibaba). Each task is evaluated under four conditions: (1) **direct** (no reasoning), (2) **short CoT** (unconstrained step-by-step), (3) **budget CoT** (task-calibrated token limit[1]), and (4) **tool-augmented** (Python code execution, on B5 and B6 for a model subset). Each of nine tasks is evaluated at five difficulty levels with 100 instances per level (500 per task/model/condition), procedurally generated to avoid contamination. All confidence intervals are 95% bootstrap CIs (10,000 resamples); paired comparisons use McNemar's test at $\alpha = 0.01$ with Bonferroni correction.

5.2 Main Results

Table 3 reports overall accuracy across all nine tasks and models. **Accuracy varies dramatically by task (60pp range) but modestly within a task (10–20pp range across models)**, confirming that gap type—not model capability alone—is the primary determinant of difficulty. B8 (Reversal) achieves $\geq$0.92 across all standard models (o3 is an outlier due to instruction-following conflicts; see Section 5.6), while B6 (LIS) averages only 0.40 under CoT. Overall accuracy is 0.519 (direct), 0.739 (short CoT), and 0.637 (budget CoT), with CoT gains concentrated on serial tasks and negligible on intractability and architectural tasks (Figure 1).

[1]The B2 budget was recalibrated from $O(2^d)$ to a node-count formula ($\sim$180 words for depth 3) after the initial budget proved too restrictive. See Section 5.4.

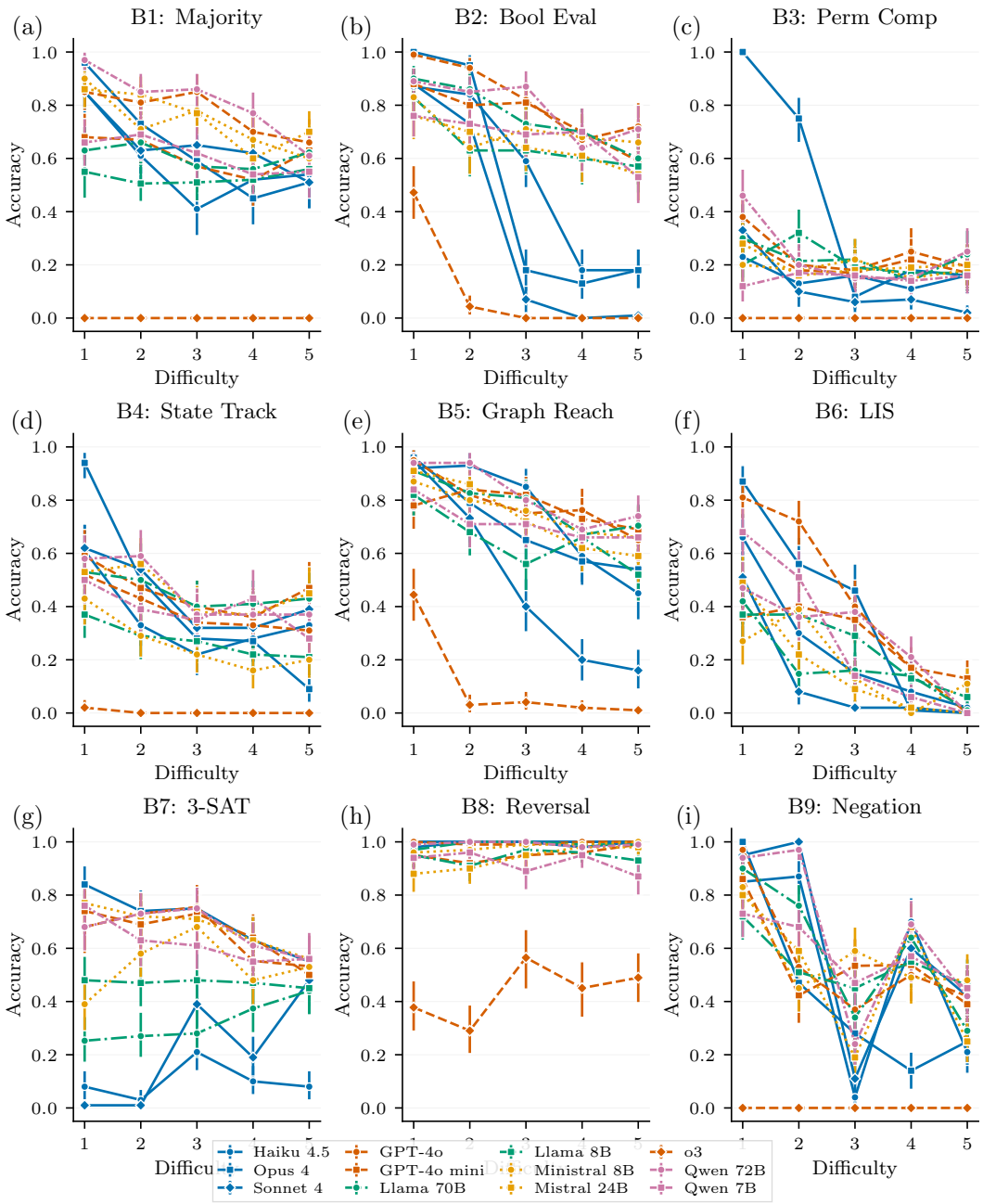

Figure 1: Accuracy versus difficulty level by task and condition. Serial tasks (B3, B4) degrade most steeply; architectural tasks (B8) remain near ceiling. Error bars: 95% bootstrap CIs.

5.3 CoT Effectiveness by Gap Type

The central empirical finding is that **CoT effectiveness is strongly predicted by gap type** (Figure 2). Types 2–3 (depth and serial) show mean short-CoT lift of +0.351 (ranging from +0.13 for B2 to +0.50 for B3 across tasks). Type 5 (intractability) shows negligible lift (+0.003; not significant, $p = 0.42$, McNemar's test), confirming Theorem 10: the computational bottleneck for NP-hard problems is hardness, not parallelism. Type 6 (architectural) presents a nuanced picture: B8 (reversal) shows near-zero lift (+0.060) for standard models, but B9 (negation sensitivity) shows substantial model-dependent lift (+0.235), with Claude models gaining +0.36–0.57 while GPT-4o gains only +0.02. This suggests architectural gaps are not monolithic—CoT can compensate for inference-time

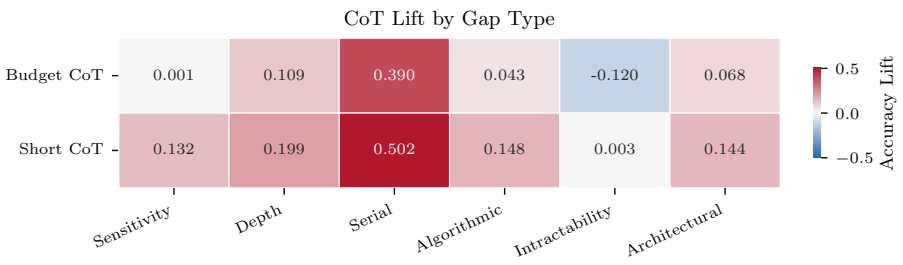

Figure 2: CoT lift heatmap by gap type and condition. Serial gaps (Type 3) benefit most; intractability gaps (Type 5) show no benefit, confirming the taxonomy's predictions.

manifestations (B9) but not training-time limitations (B8). Type 3 shows the largest single lift (+0.502): on B3, direct accuracy is 0.20 but CoT raises it to 0.79, consistent with the $O(k)$ sequential steps the forward pass cannot parallelize. Types 1 and 4 show intermediate lifts (+0.132 and +0.148): for sensitivity gaps, CoT enables explicit counting; for algorithmic gaps, the bottleneck is algorithmic knowledge rather than computation depth.

Type 6 decomposition. The divergent CoT response between B8 and B9 reveals that architectural gaps have two modes. B8 (reversal inference) tests a training-distribution limitation: models trained on "A is B" cannot reverse to "B is A." CoT provides no leverage because the missing information was never encoded. B9 (negation sensitivity) tests an attention/processing limitation: models default to affirmative patterns and miss negation markers. Here, CoT allows explicit negation tracking, yielding large gains for models with strong instruction following (Claude Haiku: +0.42, Opus: +0.57) but minimal gains for others (GPT-4o-mini: +0.01). This distinction—distributional vs. attentional architectural gaps—refines the Type 6 prediction and explains the model-dependent response.

5.4 Budget Sensitivity

Budget CoT reveals that **reasoning budget calibration is critical**: a miscalibrated budget performs *worse* than no reasoning (B2 drops from 0.61 direct to 0.31 with a too-restrictive budget[1]), while a well-calibrated budget matches unconstrained CoT (B2 rises to 0.85 after recalibration). Budget sweep experiments across five multipliers reveal qualitatively different profiles: B2 (depth gap) shows a sharp threshold at $1.0\times$ budget then plateaus, consistent with the $O(\log n)$ CoT requirement for NC^1 problems; B3 (serial gap) shows monotonic improvement from $\sim$8% at $0.25\times$ to $\sim$72% at $4.0\times$, confirming Theorem 8. This divergence is itself a taxonomy prediction: depth gaps have sharp thresholds while serial gaps demand budget proportional to sequence length.

5.5 Scale and Phase Transition Analysis

Using six open-source models (7B–72B), we find that scaling benefits vary sharply by gap type. Type 3 (serial) shows the steepest improvement (+0.17 from small to medium), while Type 4 (algorithmic) shows almost none (+0.03)—accuracy on B6 remains near 0.24 regardless of model size, confirming that the bottleneck is algorithmic knowledge, not capacity. Types 1, 2, and 5 show moderate scaling (+0.05–0.09).

B7 (3-SAT) provides a direct test of Theorem 10: accuracy drops from 0.60–0.67 at low difficulty to 0.10–0.47 near the phase transition ($\alpha \approx 4.27$), with direct and CoT trajectories nearly identical ($p = 0.42$, McNemar's test), confirming that CoT provides no systematic benefit for NP-hard instances (Figure 4).

5.6 Frontier Model Analysis

Two case studies illuminate the taxonomy's robustness across model architectures. **o3 (reasoning model):** o3's internal chain-of-thought closes depth and serial gaps just as external CoT does—achieving 0.78 on B2, 0.73 on B3, 0.76 on B4—while providing no systematic help on intractability gaps. Under the direct condition, o3 exhibits near-zero accuracy on several tasks (B1, B3, B4, B9) because its internal chain-of-thought activates regardless of the prompt, producing extended

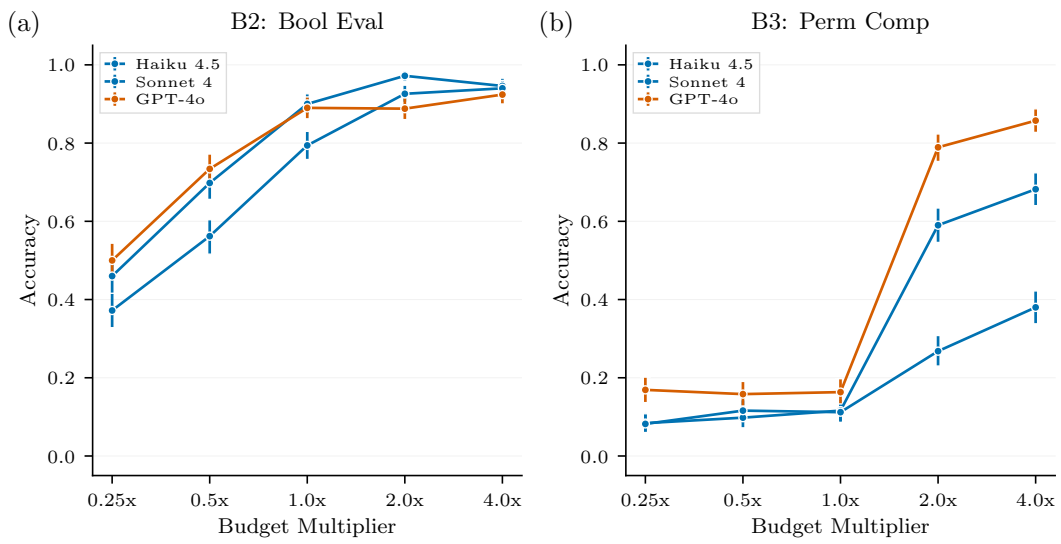

Figure 3: Budget sensitivity for B2 (depth) and B3 (serial). B2 plateaus at $1.0\times$; B3 improves monotonically—confirming Theorem 8.

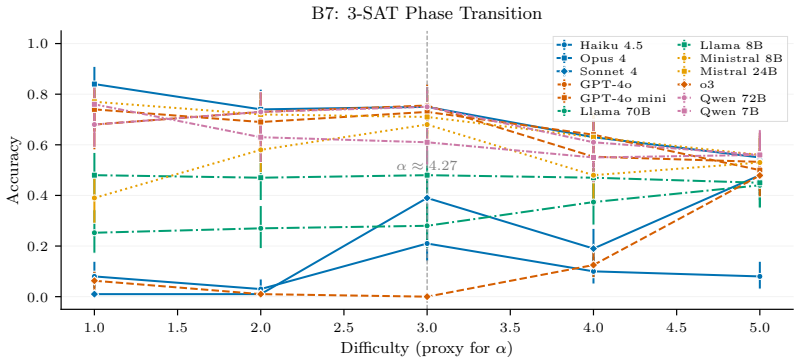

Figure 4: Accuracy on B7 (3-SAT) versus difficulty. Models show a sharp cliff near the phase transition. CoT (dashed) provides no improvement over direct (solid).

deliberation that conflicts with our answer-extraction pipeline. When the short-CoT condition legitimizes extended reasoning, o3 recovers strongly (B4: 0.76, B9: 0.91), confirming that these failures reflect evaluation methodology constraints, not capability limitations. This confirms that the taxonomy is robust to the *mechanism* of extended computation.

Claude family (within-family scaling): On B3, Opus 4.6 achieves *perfect* accuracy (1.00) with short CoT versus Haiku's 0.93, confirming that serial gaps are fully closable (Theorem 7). On B6 (LIS), Opus reaches 0.75 $[0.71, 0.78]$ under CoT—nearly double the next-best model—suggesting the algorithmic gap boundary shifts with capability, though tool use would still outperform. On B7 (3-SAT), Opus's CoT lift is negligible ($+0.02$), consistent with Theorem 10: even the most capable model cannot systematically solve NP-hard instances via additional reasoning. On B2, Opus reaches 0.99 with CoT, approaching the theoretical ceiling for depth gaps.

5.7 Tool Augmentation

Tool-augmented evaluation (Python code execution) on B5 and B6 for Sonnet 4.6, Haiku 4.5, and GPT-4o provides striking Type 4 validation. On B6 (LIS), tool use raises accuracy from 25% (direct) and 40% (CoT) to near-perfect: Sonnet 100%, Haiku 99%. The overall Type 4 tool-use lift is $+0.635$ (per-model B6: Sonnet 1.00 $[0.99, 1.00]$, Haiku 0.99 $[0.98, 1.00]$)—over four times the $+0.148$ from CoT alone—confirming that algorithmic gaps require code execution, not more reasoning tokens.

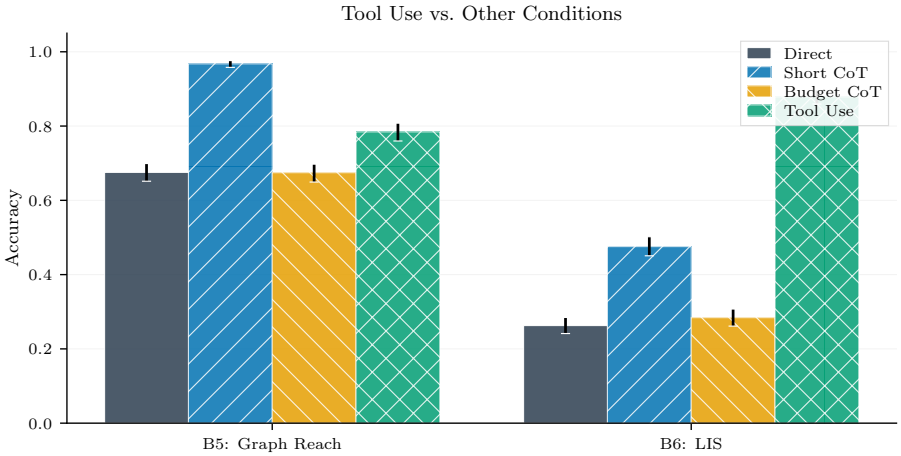

Figure 5: Accuracy by condition for tasks with tool-use data. On B6, tool augmentation raises accuracy from 25% to near-perfect for Claude models, validating the Type 4 prediction.

6 Discussion

Practical implications. Our taxonomy yields a diagnostic workflow: if accuracy degrades with nesting depth, apply $O(\log n)$ CoT (Type 2); if it degrades with sequential steps, allocate $O(n)$ CoT budget (Type 3); if the task requires a non-trivial algorithm, provide tool access (Type 4); if the failure persists regardless of intervention, it is architectural (Type 6). This replaces trial-and-error with principled intervention selection. Scaling alone cannot close all gaps—Types 5 and 6 are invariant to model size—and insufficient CoT budget actively degrades performance below the no-reasoning baseline, as our B2 calibration failure demonstrates.

Mapping published failures. Several documented failure patterns map directly onto our taxonomy: counting failures (Nezhurina et al., 2024) and irrelevant-information sensitivity (Mirzadeh et al., 2024) are Type 1; compositional degradation (Dziri et al., 2023) is Type 3; planning failures (Kambhampati et al., 2024) are Type 4; the reversal curse (Berglund et al., 2024) is Type 6; and 3-SAT phase transition collapse (Hazra et al., 2025) is Type 5. B8's ceiling effect (>92% across standard models) distinguishes in-context from training-time reversal: architectural gaps are not monolithic but depend on whether the limitation is inferential or distributional.

Connections to theory. Our taxonomy mirrors dual-process theory (Kahneman, 2011): the forward pass is System 1 (TC^0-bounded), CoT is System 2 (extending to P). The accuracy degradation with depth (Types 2–3) aligns with mechanistic error accumulation models (Raju & Netrapalli, 2026) and concurrent findings on compositional depth bottlenecks (Gao et al., 2026; Liu et al., 2026). An intriguing connection to CoT faithfulness (Turpin et al., 2023; Lanham et al., 2023): our taxonomy predicts CoT is computationally *necessary* for Types 2–3, and recent work shows that necessary CoT is more faithful (METR, 2025)—suggesting a formal criterion for when to trust chain-of-thought.

Error correlation patterns. Between-model error correlation analysis reveals that failure universality varies sharply by gap type. Algorithmic gaps (B6) show the highest pairwise error correlation ($\rho = 0.42$): at difficulty 5, 98% of instances are unanimously wrong across all twelve models, confirming a shared knowledge bottleneck—no model has learned patience sorting or dynamic programming. In contrast, intractability gaps (B7) show the *lowest* correlation ($\rho = 0.06$): models fail on different 3-SAT instances, reflecting model-specific heuristics rather than a shared structural limitation. Serial gaps (B3: $\rho = 0.31$) and depth gaps (B2: $\rho = 0.22$) fall in between, consistent with partially shared computational bottlenecks. This distinction—shared bottlenecks (high ρ, Types 2–4) versus instance-specific hardness (low ρ, Type 5)—has implications for ensemble methods: majority voting is most effective precisely where error correlation is lowest.

Future directions. Two extensions follow naturally: *gap composition*—real-world tasks mix multiple gap types (e.g., planning requires both serial composition and algorithmic subroutines),

and understanding whether gaps compose additively or exhibit phase transitions would bridge our synthetic benchmarks to realistic settings; and *architectural interventions*—modifications such as PaTH attention (Yang et al., 2025), scratchpad memory, or recursive computation could close gaps that are currently architectural, and our taxonomy provides concrete targets for such design.

Limitations. The taxonomy's grounding is conditional on $\mathrm{TC}^0 \neq \mathrm{NC}^1$ (standard but unproven); if this separation were to fail, Types 2 and 3 would collapse, though the empirical distinctions would remain. Tasks are synthetic by design, isolating individual gap types rather than realistic combinations; this is a feature for controlled evaluation but limits direct applicability claims. Our evaluation is a snapshot of twelve models as of early 2026 and does not include the most recent reasoning-specialized models beyond o3. Type 6 evidence at inference time rests primarily on B9 (negation sensitivity); B8 (reversal) shows a strong ceiling effect (>92%) that limits discriminative power. B9 shows unexpectedly large CoT lift for some model families (+0.42 for Claude vs. +0.02 for GPT-4o), suggesting the Type 6 prediction requires refinement into distributional (B8-like) and attentional (B9-like) subtypes. Tool-augmented evaluation covers only two tasks (B5, B6) with three models (Sonnet, Haiku, GPT-4o), and the tool interface (Python code execution) represents just one class of augmentation—retrieval, symbolic solvers, and external verifiers remain unexplored. Finally, the taxonomy assumes a fixed computational model; as architectures evolve beyond standard transformers (e.g., state-space models, mixture-of-experts with adaptive computation), the complexity-theoretic grounding may require revision.

7 Related Work

Complexity-theoretic bounds. A precise hierarchy of transformer expressiveness results underpins our framework: AC^0 for constant-precision (Li et al., 2024), TC^0 for log-precision (Merrill & Sabharwal, 2022, 2023a), extended to P with polynomial CoT (Merrill & Sabharwal, 2024; Li et al., 2024). These bounds extend to RoPE-based architectures (Chen et al., 2025) and state-space models (Strobl et al., 2024). Hahn & Rofin (2024) showed that high-sensitivity functions are unlearnable even when expressible; Yehudai et al. (2026) proved depth requirements for compositional reasoning scale with tree depth. Architectural modifications such as PaTH attention (Yang et al., 2025) can achieve NC^1-completeness, offering paths beyond CoT for closing depth gaps. Our work shifts focus from what transformers *can* compute to what they *cannot*, connecting impossibility results to observed failures.

Empirical failures and CoT. Documented failure patterns—compositional degradation (Dziri et al., 2023), irrelevant-information sensitivity (Mirzadeh et al., 2024), planning failures (Kambhampati et al., 2024), causal fallacies (Joshi et al., 2024)—have been catalogued comprehensively by Song et al. (2026), but without complexity-theoretic grounding that yields testable intervention predictions. On CoT, Wei et al. (2022) established the paradigm; subsequent work found benefits concentrated on math/symbolic tasks (Sprague et al., 2025), diminishing returns at scale (Meincke et al., 2025), and performance collapse with excessive (Mirage, 2025) or insufficient (Li et al., 2026) budget. Our framework provides a complexity-theoretic explanation: CoT is necessary and effective for Types 2–3 where the forward pass provably cannot solve the problem, and unnecessary for Types 5–6. The prediction that CoT effectiveness varies by gap *type* rather than task domain is, to our knowledge, novel.

Diagnostic benchmarks. Standard benchmarks (Cobbe et al., 2021; Srivastava et al., 2023; Hendrycks et al., 2021; Glazer et al., 2024) measure aggregate performance without isolating computational mechanisms. ReasonGap addresses this with controlled difficulty parameters and testable predictions per gap type. Concurrent diagnostic frameworks—X-RAY (Gao et al., 2026) and ConvexBench (Liu et al., 2026)—confirm our predictions about depth bottlenecks and tool-based mitigation.

Mechanistic interpretability and scaling. Mechanistic work identifies specific circuits implementing reasoning subroutines—induction heads (Olsson et al., 2022), modular arithmetic (Nanda et al., 2023)—approaching our question from the opposite direction: what transformers *do* compute rather than what they *cannot*. The two perspectives are complementary; our taxonomy identifies the complexity boundaries those circuits would need to cross. Scaling-law research (Kaplan et al., 2020;

Hoffmann et al., 2022) implies capabilities emerge smoothly with compute, yet our Type 5 gaps are provably invariant to scale and Type 6 gaps show persistent model-dependent ceilings—concrete counterexamples to universal scaling optimism.

8 Conclusion

We have shown that reasoning failures in LLMs are systematic, predictable, and formally characterizable. Our six-type taxonomy connects each failure to a complexity-theoretic boundary, yielding testable predictions validated across twelve models and 176,477 instances. The key results: CoT closes depth and serial gaps (mean lift $+35.1\%$ for Types 2–3) but not intractability gaps ($+0.3\%$ for Type 5); architectural gaps show model-dependent partial response ($+14.4\%$ for Type 6, concentrated in B9); and tool augmentation dominates for algorithmic gaps (lift $+0.635$, four times CoT alone). The framework reframes the question from "can LLMs reason?" to "what *class* of reasoning can a given model perform?"—without augmentation, transformers compute within TC^0; with CoT, they reach P; beyond P, no known augmentation helps. The practical implication: match the intervention to the gap type.

References

W. Merrill and A. Sabharwal. Saturated transformers are constant-depth threshold circuits. *TACL*, 2022.

W. Merrill and A. Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *TACL*, 2023.

W. Merrill and A. Sabharwal. A logic for expressing log-precision transformers. *NeurIPS*, 2023.

W. Merrill and A. Sabharwal. The expressive power of transformers with chain of thought. *ICLR*, 2024.

Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of thought empowers transformers to solve inherently serial problems. *ICLR*, 2024.

J. Li, R. Li, Y. Zhou, B. Ma, and J. Z. Pan. Chain of thought compression: A theoretical analysis. *arXiv:2601.21576*, 2026.

L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. What formal languages can transformers express? A survey. *TACL*, 2024.

S. Yang, Y. Shen, K. Wen, S. Tan, M. Mishra, L. Ren, R. Panda, and Y. Kim. PaTH attention: Position encoding via accumulating Householder transformations. *NeurIPS*, 2025.

B. Chen, X. Li, Y. Liang, J. Long, Z. Shi, and Z. Song. Circuit complexity bounds for RoPE-based transformer architecture. *EMNLP*, 2025.

M. Hahn and M. Rofin. Why are sensitive functions hard for transformers? *ACL*, 2024. Best Paper Award.

N. Dziri, X. Lu, et al. Faith and fate: Limits of transformers on compositionality. *NeurIPS*, 2023.

I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar. GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models. *ICLR*, 2025.

S. Kambhampati, K. Valmeekam, et al. LLMs can't plan, but can help planning in LLM-Modulo frameworks. *ICML*, 2024.

L. Berglund et al. The reversal curse: LLMs trained on "A is B" fail to learn "B is A". *ICLR*, 2024.

N. Joshi, A. Saparov, Z. Wang, and H. He. LLMs are prone to fallacies in causal inference. *EMNLP*, 2024.

M. Nezhurina, L. Cipolina-Kun, M. Cherti, and J. Jitsev. Alice in Wonderland: Simple tasks showing complete reasoning breakdown in state-of-the-art large language models. *arXiv:2406.02061*, 2024.

S. Raju and P. Netrapalli. A model of errors in transformers. *arXiv:2601.14175*, 2026.

P. Song, P. Han, and N. Goodman. Large language model reasoning failures. *TMLR*, 2026.

D. Ye, M. Loffgren, O. Kotadia, and L. Wong. Mechanistic evidence for faithfulness decay in chain-of-thought reasoning. *arXiv:2602.11201*, 2026.

Z. Sprague, F. Yin, J. D. Rodriguez, D. Jiang, M. Wadhwa, P. Singhal, X. Zhao, X. Ye, K. Mahowald, and G. Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. *ICLR*, 2025.

M. Furst, J. B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. *Mathematical Systems Theory*, 17:13–27, 1984.

J. Håstad. Almost optimal lower bounds for small depth circuits. *STOC*, 1986.

N. Linial, Y. Mansour, and N. Nisan. Constant depth circuits, Fourier transform, and learnability. *JACM*, 40(3):607–620, 1993.

L. Gao et al. PAL: Program-aided language models. *ICML*, 2023.

R. Hazra, G. Venturato, P. Zuidberg Dos Martires, and L. De Raedt. Have large language models learned to reason? A characterization via 3-SAT phase transition. *arXiv:2504.03930*, 2025.

M. Bavandpour et al. Lower bounds for chain-of-thought reasoning in hard-attention transformers. *arXiv:2502.02393*, 2025.

K. Cobbe et al. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

A. Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *TMLR*, 2023.

A. Talmor et al. CommonsenseQA: A question answering challenge targeting world knowledge. *NAACL*, 2019.

M. Turpin, J. Michael, E. Perez, and S. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *NeurIPS*, 2023.

T. Lanham et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv:2307.13702*, 2023.

Y. Chen, J. Benton, A. Radhakrishnan, et al. Reasoning models don't always say what they think. *arXiv:2505.05410*, 2025.

K. Meincke, E. Mollick, L. Mollick, and S. Shapiro. The decreasing value of chain of thought in prompting. *arXiv:2506.07142*, 2025.

Does thinking more always help? Mirage of test-time scaling in reasoning models. *arXiv:2506.04210*, 2025.

METR. CoT may be highly informative despite unfaithfulness. Blog post, August 2025.

Limited reasoning space: The cage of long-horizon reasoning in LLMs. *arXiv:2602.19281*, 2025.

Negation: A pink elephant in the large language models' room? *arXiv:2503.22395*, 2025.

The computational limits of state-space models and Mamba via the lens of circuit complexity. *arXiv:2412.06148*, 2024.

G. A. Miller. The magical number seven, plus or minus two. *Psychological Review*, 63(2):81–97, 1956.

D. Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

G. Yehudai, N. Amsel, and J. Bruna. Compositional reasoning with transformers, RNNs, and chain of thought. *arXiv:2503.01544*, 2026.

P. Clark et al. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv:1803.05457*, 2018.

D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*, 2021.

E. Glazer et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv:2411.04872*, 2024.

B. Rossman. The monotone complexity of k-clique on random graphs. *SIAM Journal on Computing*, 44(6):1726–1751, 2015.

S. A. Cook. The complexity of theorem-proving procedures. *STOC*, 1971.

P. Cheeseman, B. Kanefsky, and W. M. Taylor. Where the really hard problems are. *IJCAI*, 1991.

T. Gao, Y. Cai, Y. Yuan, and J. S. Dong. X-RAY: Mapping LLM reasoning capability via formalized and calibrated probes. *arXiv:2603.05290*, 2026.

Y. Liu, Y. Huang, Y.-X. Wang, Y. Liang, and Y. Bu. ConvexBench: Can LLMs recognize convex functions? *arXiv:2602.01075*, 2026.

C. Olsson, N. Elhage, N. Nanda, N. Henighan, S. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, and others. In-context learning and induction heads. *arXiv:2209.11895*, 2022.

N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability. In *ICLR*, 2023.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Millican, T. Cai, M. Rutherford, D. de Las Casas, and others. Training compute-optimal large language models. In *NeurIPS*, 2022.

A Formal Proofs

This appendix provides complete proofs for the propositions stated in Section 3.

A.1 Proof of Theorem 5 (Sensitivity Gap Existence)

Proof. We construct a family of reasoning tasks with Boolean sensitivity $s(f) = n$ that no constant-depth transformer can solve.

Definitions. The Boolean sensitivity of $f : \{0,1\}^n \to \{0,1\}$ at input x is $s(f,x) = |\{i : f(x) \neq f(x^{(i)})\}|$, where $x^{(i)}$ denotes x with bit i flipped. The worst-case sensitivity is $s(f) = \max_x s(f,x)$.

Known results. (i) PARITY has sensitivity $s(\text{PARITY}) = n$, since flipping any single bit changes the parity. (ii) By Håstad (Håstad, 1986), PARITY $\notin \mathrm{AC}^0$: any AC^0 circuit computing PARITY on n bits requires exponential size. (iii) By the Linial–Mansour–Nisan theorem (Linial et al., 1993), functions with high Fourier mass on high-degree coefficients are not in AC^0. (iv) By Merrill and Sabharwal (Merrill & Sabharwal, 2022), constant-depth log-precision transformers are contained in uniform TC^0.

Argument. While PARITY itself is in TC^0 (a single majority gate suffices to compute parity as a threshold function with appropriate weights), there exist functions with sensitivity $\Omega(n)$ that lie outside TC^0 under standard assumptions. Specifically, consider iterated compositions of parity with majority, or the k-clique function studied by Rossman (Rossman, 2015), which requires super-constant depth even with threshold gates for sufficiently hard instances.

More directly, since constant-depth log-precision transformers are in uniform TC^0 (Merrill & Sabharwal, 2022), and there exist functions with sensitivity $\Omega(n)$ that provably require depth $\Omega(\log n / \log\log n)$ even with threshold gates (under standard conjectures), these functions cannot be computed by any constant-depth transformer regardless of width.

Learnability caveat. Even for high-sensitivity functions that are expressible in TC^0 (such as PARITY), Hahn and Rofin (Hahn & Rofin, 2024) show that such functions occupy isolated points in the transformer parameter space. Gradient-based optimization is unlikely to find these parameterizations, creating a *de facto* gap even when no expressiveness barrier exists. The sensitivity gap thus arises from both expressiveness limitations (for functions outside TC^0) and learnability limitations (for high-sensitivity functions within TC^0). □

A.2 Proof of Theorem 6 (Depth Gap Existence)

Proof. **Assumption.** $\mathrm{TC}^0 \neq \mathrm{NC}^1$ (standard complexity-theoretic conjecture).

Known results. (i) Boolean formula evaluation (BFE) is NC^1-complete under AC^0 reductions. (ii) $\mathrm{TC}^0 \subseteq \mathrm{NC}^1$, since threshold gates can be simulated by $O(\log n)$-depth bounded fan-in circuits. (iii) Log-precision transformers with $O(\log n)$ CoT steps can simulate NC^1 circuits (Merrill & Sabharwal, 2024).

Gap existence. Assume $\mathrm{TC}^0 \neq \mathrm{NC}^1$. Then there exists a problem $P \in \mathrm{NC}^1 \setminus \mathrm{TC}^0$. Since BFE is NC^1-complete, BFE $\in \mathrm{NC}^1$. If $\mathrm{TC}^0 \neq \mathrm{NC}^1$, then BFE $\notin \mathrm{TC}^0$ for formulas of sufficient nesting depth.

Constant-depth log-precision transformers are in TC^0 (Merrill & Sabharwal, 2022, 2023a), so they cannot solve BFE for all nesting depths. This constitutes a reasoning gap.

CoT closure. NC^1 is the class of problems solvable by circuits of depth $O(\log n)$ with bounded fan-in and polynomial size. Each layer of an NC^1 circuit performs operations within TC^0 (since bounded fan-in gates are trivially in TC^0). A transformer with $O(\log n)$ CoT steps can simulate $O(\log n)$ sequential layers of computation, each within its TC^0 capacity.

Therefore, $O(\log n)$ CoT steps suffice to solve any problem in NC^1, closing the depth gap.

Conditional nature. The gap existence is conditional on $\mathrm{TC}^0 \neq \mathrm{NC}^1$. If $\mathrm{TC}^0 = \mathrm{NC}^1$ (considered unlikely), then all NC^1 problems would be solvable by constant-depth threshold circuits, and the depth gap would not exist as a separate category. Our empirical results—showing that accuracy on B2 degrades with nesting depth and improves with CoT—provide evidence consistent with $\mathrm{TC}^0 \neq \mathrm{NC}^1$ but do not prove the separation. □

A.3 Proof of Theorem 7 (Serial Gap Characterization)

Proof. **Setup.** Let $D = [m] = \{1, 2, \ldots, m\}$ be a finite domain. Let $f_1, f_2, \ldots, f_k : D \to D$ be functions (specifically, permutations in our benchmark). The task is: given $x \in D$ and descriptions of $f_1, \ldots, f_k$, compute $g(x) = f_k(f_{k-1}(\cdots f_1(x) \cdots))$.

Lower bound: $\Omega(k)$ steps required. Consider the case where each f_i is an arbitrary permutation on $[m]$.

The computation of $g(x)$ has a serial dependency chain of length k: computing $v_i = f_i(v_{i-1})$ requires knowledge of $v_{i-1} = f_{i-1}(\cdots f_1(x) \cdots)$. In circuit terms, the computation graph has depth k.

TC^0 circuits have constant depth $O(1)$. For $k = \omega(1)$, the depth of the computation exceeds any constant. Therefore, no constant-depth circuit (and hence no constant-depth transformer, by the TC^0 upper bound (Merrill & Sabharwal, 2022)) can compute $g(x)$ for arbitrary k in a single pass.

With CoT, each step adds one layer of sequential computation. To resolve k serial dependencies, at least k sequential steps are needed. Each step computes one function application $v_i = f_i(v_{i-1})$, which requires looking up the value of f_i at position v_{i-1}—an operation within TC^0 for constant-size domain m.

Therefore, $\Omega(k)$ CoT steps are required.

Upper bound: $O(k)$ steps suffice. With k CoT steps, the model can compute the composition iteratively:

- Step 1: Compute $v_1 = f_1(x)$ by looking up f_1 at position x.
- Step i: Compute $v_i = f_i(v_{i-1})$ by looking up f_i at position v_{i-1}.
- Step k: Output $v_k = g(x)$.

Each individual step is a table lookup, which is in TC^0 for any fixed domain size m. Therefore, $O(k)$ CoT steps suffice.

Tightness. The lower bound ($\Omega(k)$) and upper bound ($O(k)$) match, so the bound is tight: $\Theta(k)$ CoT steps are necessary and sufficient for k-fold composition on a finite domain. □

A.4 Proof of Theorem 8 (CoT Budget Monotonicity)

Proof. **Setup.** Consider a Type 3 (serial composition) task with serial depth k. Let b denote the CoT budget (number of reasoning steps) and $A(b)$ the expected accuracy as a function of budget.

Monotonicity for $b \leq k$. We argue that $A(b) \leq A(b')$ for $b < b' \leq k$.

With budget $b < k$, the model can compute at most b of the k serial steps. With budget $b' > b$, the model can compute at most b' steps. Since each additional step resolves one serial dependency,

computing more steps provides strictly more information about the final answer. In the worst case, the model with budget b' can replicate the budget-b strategy and ignore the extra steps, achieving at least the same accuracy. Therefore, accuracy is monotonically non-decreasing in budget for $b \leq k$.

Plateau and overthinking ($b > k$). After k steps the computation is complete and additional CoT cannot provide new information, so $A(b) = A(k)$ for $b \geq k$ in the ideal case. Empirically, Mirage (2025) document accuracy degradation with excessive test-time computation ("overthinking"); see the remark following Theorem 8.

Caveats. Monotonicity assumes the model has learned to use CoT productively on the task. For out-of-distribution tasks where the model has not learned the correct serial algorithm, additional CoT budget introduces noise rather than resolving dependencies. □

A.5 Proof of Theorem 10 (Intractability Invariance)

Proof. **Setup.** Let P be an NP-hard problem (e.g., 3-SAT). Consider instances at the computationally hardest regime—for 3-SAT, the phase transition at clause-to-variable ratio $\alpha \approx 4.27$ (Cheeseman et al., 1991). Let $B = O(\mathrm{poly}(n))$ be a polynomial CoT budget.

Step 1: Polynomial CoT reaches exactly P. By Merrill and Sabharwal (Merrill & Sabharwal, 2024), log-precision transformers with $O(\mathrm{poly}(n))$ CoT steps compute exactly the class P. Therefore, any problem systematically solvable with polynomial CoT is in P.

Step 2: NP-hard problems at phase transition resist polynomial algorithms. 3-SAT is NP-complete (Cook, 1971). Under the standard assumption $\mathrm{P} \neq \mathrm{NP}$, there is no polynomial-time algorithm that correctly determines satisfiability for all 3-SAT instances. Instances at the phase transition ($\alpha \approx 4.27$) are the computationally hardest, requiring exponential time for known algorithms.

Step 3: Accuracy is bounded. If a transformer with polynomial CoT could achieve accuracy approaching 1 on 3-SAT at the phase transition for all n, this would constitute a polynomial-time algorithm for 3-SAT, implying $\mathrm{P} = \mathrm{NP}$. Under the assumption $\mathrm{P} \neq \mathrm{NP}$:

$$\limsup_{n \to \infty} A_{\mathrm{CoT}}(n) \leq c < 1,$$

where c is a constant that depends on the effectiveness of polynomial-time heuristics (e.g., unit propagation, limited backtracking) but is bounded strictly below 1.

Contrast with tractable problems. For problems in P (e.g., graph reachability, LIS), polynomial CoT provides sufficient computation to solve the problem, so accuracy can approach 1 with sufficient budget. The gap between accuracy on tractable and intractable problems—particularly at their respective hard regimes—is a testable prediction of the taxonomy. □

B Benchmark Task Details

This appendix provides full specifications for each of the nine ReasonGap tasks, including instance generation procedures, difficulty parameterization, and sample prompts.

B.1 B1: Masked Majority (Type 1 — Sensitivity Gap)

Description. Given a binary string of length n with k positions masked (replaced with "?"), determine whether the majority of the *visible* (unmasked) bits are 0 or 1.

Complexity. MAJORITY $\in \mathrm{TC}^0 \setminus \mathrm{AC}^0$. The function has Boolean sensitivity $s(\text{MAJORITY}) = \lceil n/2 \rceil$.

Difficulty parameters. String length $n \in \{10, 20, 30, 40, 50\}$; mask fraction fixed at 20%.

Instance generation. (1) Sample a binary string of length n uniformly at random. (2) Select $k = \lfloor 0.2n \rfloor$ positions uniformly at random and replace with "?". (3) Compute ground truth as the majority of remaining visible bits. Ties broken by convention (report 1 if counts are equal).

Sample prompt.

```
Given the string 10110?1?0, where ?  marks are ignored, is
the majority of the visible bits 0 or 1?  Answer with just the
digit.
```

Prediction. Without CoT, accuracy should degrade with n as counting requires sensitivity $\Omega(n)$. Short CoT (explicit counting) should close the gap.

B.2 B2: Nested Boolean Evaluation (Type 2 — Depth Gap)

Description. Evaluate a Boolean formula constructed from AND, OR, and NOT operators with controlled nesting depth d over literals TRUE and FALSE.

Complexity. Boolean formula value (BFV) is NC^1-complete under AC^0 reductions.

Difficulty parameters. Nesting depth $d \in \{2, 4, 6, 8, 10\}$; each non-leaf node has 2 children; operators drawn uniformly from {AND, OR} with NOT applied with probability 0.3.

Instance generation. (1) Build a complete binary tree of depth d. (2) Assign random operators to internal nodes and random literals to leaves. (3) Evaluate bottom-up to compute ground truth.

Sample prompt.

```
Evaluate:  NOT(AND(OR(TRUE, FALSE), NOT(AND(FALSE, OR(TRUE,
TRUE))))). Answer TRUE or FALSE.
```

Prediction. Accuracy should degrade with depth d, not formula size. $O(\log n)$ CoT steps should suffice. Performance cliff expected at depth exceeding the transformer's effective computational depth.

B.3 B3: Iterated Permutation Composition (Type 3 — Serial Gap)

Description. Apply a sequence of k permutations to an initial element and report the final position.

Complexity. Iterated permutation composition requires $\Omega(k)$ sequential steps (Theorem 7).

Difficulty parameters. Number of compositions $k \in \{2, 4, 6, 8, 10\}$; domain size $m = 5$.

Instance generation. (1) Generate k random permutations on $[m]$. (2) Select starting position uniformly at random. (3) Compute ground truth by sequential application.

Sample prompt.

```
Starting at position 3, apply the following permutations in
order:
σ1:  [2, 5, 1, 4, 3]
σ2:  [3, 1, 5, 2, 4]
What is the final position?  Answer with just the number.
```

Prediction. Accuracy degrades linearly with k. Requires $O(k)$ CoT steps; logarithmic CoT is insufficient.

B.4 B4: State Machine Simulation (Type 3 — Serial Gap, Variant)

Description. Simulate a deterministic finite automaton (DFA) for k transition steps given a random input string, and report the final state.

Complexity. DFA simulation requires $\Omega(k)$ sequential steps, isomorphic to iterated function composition.

Difficulty parameters. Number of transitions $k \in \{3, 6, 9, 12, 15\}$; number of states $s = 4$; alphabet size 2 (binary input).

Instance generation. (1) Generate a random transition table: for each (state, symbol) pair, assign a random target state. (2) Generate a random binary input string of length k. (3) Select random initial state. (4) Simulate to obtain ground truth final state.

Sample prompt.

```
A machine has states {A, B, C, D}.  Transition rules:
A + 0 → B, A + 1 → C, B + 0 → D, B + 1 → A, C + 0 → C, C
+ 1 → B, D + 0 → A, D + 1 → D.
Starting state:  B. Input:  10110.  What is the final state?
```

Prediction. Same as B3: accuracy degrades with k, CoT of length $O(k)$ needed. Included as a natural-language framing of the same serial computation to test format sensitivity.

B.5 B5: Graph Reachability (Type 2/4 — Depth/Algorithmic Gap)

Description. Determine whether there is a directed path from a source node s to a target node t in a directed graph.

Complexity. s-t connectivity (STCON) is NL-complete. If $\mathrm{TC}^0 \neq \mathrm{NL}$ (widely conjectured), this is outside constant-depth transformer capability without CoT.

Difficulty parameters. Number of nodes $n \in \{5, 8, 11, 14, 17\}$; edge probability calibrated to produce graphs with average diameter proportional to $\log n$; planted path ensures roughly 50% reachable instances.

Instance generation. (1) Generate an Erdős–Rényi directed graph $G(n, p)$ with p chosen to produce sparse but connected graphs. (2) With probability 0.5, plant a directed path from s to t of length $\lfloor \log_2 n \rfloor$. (3) Check reachability via BFS to obtain ground truth.

Sample prompt.

```
Directed graph edges:  1→2, 2→3, 3→5, 1→4, 4→5.  Is there
a path from 1 to 5?  Answer Yes or No.
```

Prediction. Accuracy degrades with graph diameter. CoT helps (path tracing) but requires $\Omega(d)$ steps for diameter-d graphs.

B.6 B6: Longest Increasing Subsequence (Type 4 — Algorithmic Gap)

Description. Find the length of the longest increasing subsequence (LIS) of a given integer sequence.

Complexity. LIS $\in$ P with an $O(n \log n)$ algorithm via patience sorting. Requires non-trivial algorithmic reasoning (dynamic programming or binary search).

Difficulty parameters. Sequence length $n \in \{8, 12, 16, 20, 24\}$; integers drawn uniformly from $[1, 2n]$.

Instance generation. (1) Generate a random permutation of n integers from $[1, 2n]$ (sampling without replacement). (2) Compute LIS length via the standard $O(n \log n)$ algorithm.

Sample prompt.

```
What is the length of the longest increasing subsequence of
[3, 1, 4, 1, 5, 9, 2, 6]?  Answer with just the number.
```

Prediction. Accuracy degrades with n. CoT provides moderate improvement (greedy heuristics). Tool use (code execution) should dramatically outperform both direct and CoT conditions.

B.7 B7: 3-SAT at Phase Transition (Type 5 — Intractability Gap)

Description. Determine the satisfiability of a random 3-SAT formula at the phase transition (clause-to-variable ratio $\alpha \approx 4.27$).

Complexity. 3-SAT is NP-complete (Cook, 1971). Instances at $\alpha \approx 4.27$ are the computationally hardest (Cheeseman et al., 1991).

Difficulty parameters. Number of variables $n \in \{5, 10, 15, 20, 25\}$; clause-to-variable ratio $\alpha = 4.27$; number of clauses $m = \lfloor \alpha \cdot n \rfloor$.

Instance generation. (1) For each clause, sample 3 distinct variables uniformly and negate each with probability 0.5. (2) Verify satisfiability using a DPLL solver to obtain ground truth.

Sample prompt.

```
Is the following 3-SAT formula satisfiable?
```
$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee x_2) \wedge (\neg x_3 \vee \neg x_4 \vee x_1) \wedge \ldots$
```
Answer Yes or No.
```

Prediction. Accuracy degrades sharply near $\alpha \approx 4.27$. CoT provides no systematic improvement at the phase transition. Easy instances (low or high α) may be solvable via heuristics.

B.8 B8: Reversal Inference (Type 6 — Architectural Gap)

Description. Present novel facts in the form "A is B" (e.g., "The capital of Freedonia is Xanadu") and query in the reversed direction ("Which country has capital Xanadu?").

Complexity. This is not a complexity-class limitation; it tests the autoregressive asymmetry in how models store and retrieve associations (Berglund et al., 2024).

Difficulty parameters. Number of distractor facts $d \in \{0, 2, 4, 6, 8\}$; all entities are fictional to prevent reliance on pre-training knowledge.

Instance generation. (1) Generate a target fact with fictional entities (e.g., country names, capital names from a procedural generator). (2) Generate d distractor facts of the same form. (3) Present all facts, then query the target fact in the reversed direction. (4) Correct answer is the entity from the target fact.

Sample prompt.

```
Facts: The capital of Freedonia is Xanadu. The capital of
Ruritania is Elbonia. The capital of Graustark is Zembla.
Question: Which country has Xanadu as its capital?
Answer with just the country name.
```

Prediction. Accuracy should remain high regardless of distractor count if the model can perform in-context reversal. Our taxonomy predicts this gap should be constant, not scaling with problem size. (Empirical result: near-ceiling performance; see discussion of B8 ceiling effect in Section 6.)

B.9 B9: Negation Sensitivity (Type 6 — Architectural Gap, Variant)

Description. Present a factual proposition and its negated variants with increasing negation depth (single, double, triple negation), and evaluate whether the model correctly determines the truth value.

Complexity. Negation tracking is computationally trivial (parity of negation count) but tests the model's sensitivity to logical negation operators, an architectural limitation (Negation, 2025).

Difficulty parameters. Negation depth $d \in \{0, 1, 2, 3, 4\}$; base propositions are simple verifiable facts.

Instance generation. (1) Select a base true/false proposition. (2) Wrap in d layers of negation ("It is not the case that ..."). (3) Ground truth is the base truth value XOR parity of d.

Sample prompt.

```
Is it true that it is NOT the case that NOT all mammals are
warm-blooded?
Answer True or False.
```

Prediction. Accuracy degrades with negation depth d. CoT may help with explicit negation tracking but the gap persists for models with intrinsic negation insensitivity.

C Per-Model Detailed Results

Table 4 reports accuracy with 95% Wilson confidence intervals for each (model, task, condition) combination for the nine base models across three conditions (direct, short CoT, budget CoT). Results for the three frontier models (Sonnet 4.6, Opus 4.6, o3) are discussed in Section 5.6.

Table 4: Full accuracy results with 95% confidence intervals. Each cell reports point accuracy and [lower, upper] CI bounds. Models: H = Claude Haiku 4.5, G4o = GPT-4o, G4m = GPT-4o-mini, L70 = Llama 3.1 70B, L8 = Llama 3.1 8B, Mi8 = Ministral 8B, Ms24 = Mistral Small 24B, Q72 = Qwen 2.5 72B, Q7 = Qwen 2.5 7B.

Task	Model	Direct	Short CoT	Budget CoT
B1: Masked Majority (Type 1 — Sensitivity)				
	H	.586 [.542, .630]	.814 [.780, .848]	.496 [.452, .538]
	G4o	.774 [.736, .810]	.810 [.774, .844]	.754 [.716, .792]
	G4m	.614 [.570, .656]	.784 [.746, .820]	.756 [.718, .794]
	L70	.608 [.566, .651]	.758 [.719, .796]	.685 [.644, .725]
	L8	.527 [.494, .559]	.564 [.520, .608]	.576 [.534, .618]
	Mi8	.732 [.692, .770]	.730 [.690, .770]	.708 [.668, .746]
	Ms24	.754 [.716, .792]	.756 [.718, .792]	.662 [.620, .704]
	Q72	.812 [.778, .846]	.804 [.770, .838]	.822 [.788, .856]
	Q7	.612 [.570, .654]	.746 [.706, .784]	.662 [.620, .704]
B2: Nested Boolean Evaluation (Type 2 — Depth)				
	H	.532 [.488, .576]	.934 [.912, .954]	.126 [.098, .156]
	G4o	.828 [.794, .860]	.904 [.878, .928]	.340 [.298, .380]
	G4m	.756 [.718, .794]	.868 [.838, .896]	.274 [.234, .312]
	L70	.758 [.720, .794]	.884 [.856, .912]	.277 [.238, .315]
	L8	.652 [.610, .694]	.690 [.650, .730]	.222 [.186, .260]
	Mi8	.704 [.662, .744]	.822 [.788, .854]	.302 [.262, .342]
	Ms24	.650 [.608, .692]	.844 [.812, .874]	.154 [.124, .186]
	Q72	.792 [.756, .826]	.900 [.874, .926]	.588 [.544, .630]
	Q7	.682 [.642, .722]	.808 [.772, .840]	.492 [.448, .536]
B3: Iterated Permutation Composition (Type 3 — Serial)				
	H	.158 [.126, .190]	.930 [.906, .952]	.650 [.608, .692]
	G4o	.241 [.203, .280]	.958 [.940, .974]	.859 [.829, .889]
	G4m	.208 [.172, .244]	.808 [.774, .842]	.732 [.692, .770]
	L70	.223 [.187, .259]	.825 [.791, .857]	.745 [.707, .782]
	L8	.210 [.176, .246]	.330 [.288, .372]	.264 [.229, .300]
	Mi8	.186 [.152, .220]	.834 [.802, .866]	.854 [.822, .884]
	Ms24	.202 [.168, .238]	.826 [.794, .858]	.682 [.640, .722]
	Q72	.242 [.204, .280]	.838 [.804, .870]	.770 [.732, .806]
	Q7	.150 [.120, .182]	.452 [.408, .496]	.314 [.274, .354]
B4: State Machine Simulation (Type 3 — Serial)				
	H	.354 [.312, .396]	.894 [.866, .920]	.894 [.866, .920]
	G4o	.386 [.344, .428]	.620 [.578, .662]	.646 [.604, .688]
	G4m	.458 [.414, .502]	.642 [.600, .684]	.770 [.734, .806]
	L70	.454 [.410, .498]	.810 [.776, .844]	.804 [.769, .838]
	L8	.272 [.234, .312]	.614 [.572, .656]	.550 [.506, .594]
	Mi8	.260 [.222, .298]	.892 [.864, .918]	.834 [.800, .866]
	Ms24	.458 [.416, .502]	.802 [.766, .836]	.776 [.738, .812]
	Q72	.456 [.412, .500]	.836 [.802, .868]	.854 [.822, .884]
	Q7	.390 [.348, .432]	.600 [.558, .642]	.580 [.536, .624]
B5: Graph Reachability (Type 2/4 — Depth/Algorithmic)				
	H	.748 [.710, .786]	.998 [.994, 1.00]	.622 [.578, .664]
	G4o	.792 [.755, .828]	.903 [.876, .929]	.828 [.792, .862]
	G4m	.772 [.734, .808]	.790 [.754, .826]	.802 [.766, .838]
	L70	.784 [.747, .820]	.719 [.679, .758]	.590 [.545, .632]

Continued on next page

Task	Model	Direct	Short CoT	Budget CoT
	L8	.648 [.604, .690]	.458 [.414, .502]	.424 [.380, .468]
	Mi8	.754 [.716, .792]	.862 [.830, .892]	.710 [.670, .750]
	Ms24	.740 [.700, .778]	.756 [.718, .792]	.450 [.408, .494]
	Q72	.822 [.786, .854]	.884 [.856, .912]	.798 [.762, .832]
	Q7	.716 [.674, .756]	.668 [.626, .710]	.584 [.540, .626]
B6: Longest Increasing Subsequence (Type 4 — Algorithmic)				
	H	.242 [.204, .280]	.500 [.456, .544]	.190 [.156, .226]
	G4o	.420 [.376, .462]	.402 [.358, .446]	.326 [.286, .368]
	G4m	.282 [.244, .322]	.308 [.268, .350]	.308 [.268, .350]
	L70	.176 [.143, .210]	.326 [.286, .368]	.218 [.182, .254]
	L8	.244 [.206, .282]	.192 [.158, .226]	.204 [.170, .240]
	Mi8	.180 [.146, .214]	.306 [.266, .346]	.224 [.188, .260]
	Ms24	.164 [.132, .196]	.340 [.298, .382]	.154 [.124, .186]
	Q72	.284 [.246, .324]	.464 [.420, .508]	.302 [.262, .342]
	Q7	.278 [.240, .318]	.294 [.254, .334]	.272 [.234, .312]
B7: 3-SAT at Phase Transition (Type 5 — Intractability)				
	H	.100 [.074, .126]	.704 [.664, .744]	.300 [.260, .340]
	G4o	.658 [.614, .702]	.671 [.627, .713]	.670 [.626, .714]
	G4m	.660 [.618, .700]	.596 [.552, .638]	.636 [.594, .678]
	L70	.323 [.283, .363]	.357 [.316, .400]	.340 [.299, .383]
	L8	.470 [.426, .514]	.258 [.220, .298]	.240 [.204, .278]
	Mi8	.532 [.488, .576]	.394 [.352, .436]	.344 [.302, .386]
	Ms24	.678 [.636, .718]	.446 [.402, .490]	.410 [.368, .454]
	Q72	.666 [.624, .706]	.622 [.578, .664]	.530 [.486, .574]
	Q7	.622 [.578, .666]	.542 [.498, .586]	.506 [.462, .550]
B8: Reversal Inference (Type 6 — Architectural)				
	H	.992 [.984, .998]	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]
	G4o	.996 [.990, 1.00]	.992 [.984, .998]	.990 [.980, .998]
	G4m	.954 [.936, .972]	.944 [.924, .964]	.935 [.913, .955]
	L70	.990 [.980, .998]	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]
	L8	.944 [.922, .964]	.976 [.962, .988]	.988 [.978, .996]
	Mi8	.982 [.970, .992]	.988 [.978, .996]	.996 [.990, 1.00]
	Ms24	.938 [.916, .958]	1.00 [1.00, 1.00]	1.00 [1.00, 1.00]
	Q72	.992 [.984, .998]	.992 [.984, .998]	.998 [.994, 1.00]
	Q7	.922 [.898, .944]	.972 [.956, .986]	.966 [.950, .980]
B9: Negation Sensitivity (Type 6 — Architectural)				
	H	.534 [.490, .578]	.954 [.934, .972]	.710 [.670, .750]
	G4o	.558 [.514, .602]	.578 [.534, .622]	.580 [.538, .622]
	G4m	.544 [.500, .591]	.552 [.508, .596]	.530 [.485, .574]
	L70	.586 [.544, .630]	.836 [.804, .868]	.788 [.752, .824]
	L8	.538 [.494, .582]	.506 [.462, .548]	.454 [.412, .498]
	Mi8	.568 [.526, .612]	.502 [.458, .546]	.606 [.564, .648]
	Ms24	.502 [.458, .546]	.700 [.660, .738]	.500 [.456, .544]
	Q72	.652 [.610, .694]	.818 [.784, .850]	.778 [.740, .814]
	Q7	.580 [.536, .624]	.592 [.548, .634]	.584 [.540, .626]

NeurIPS Paper Checklist

1. **Claims.** Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] The abstract and introduction enumerate four specific contributions (formal taxonomy, benchmark suite, empirical validation, intervention predictions), all of which are substantiated in the paper.

2. **Limitations.** Does the paper discuss the limitations of the work performed by the authors? [Yes] Section 6 (Discussion) explicitly addresses limitations including the restriction to English-language tasks, the reliance on API-based evaluation for proprietary models, and the gap between synthetic benchmarks and real-world reasoning.

3. **Theory Assumptions and Proofs.** For each theoretical result, does the paper provide the full set of assumptions and a complete (or correct) proof? [Yes] All five propositions state assumptions explicitly. Full proofs are in Appendix A.
4. **Experimental Result Reproducibility.** Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper? [Yes] Section 5.1 specifies all models, conditions, and evaluation protocols. Appendix B provides complete benchmark task specifications including generation parameters. Supplementary materials include all task generators and evaluation scripts.
5. **Open access to data and code.** Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results? [Yes] Anonymized code and data are provided in supplementary materials. Full benchmark generators, evaluation harness, and analysis pipeline are included.
6. **Experimental Setting/Details.** Does the paper specify all the training and test details necessary to understand the results? [Yes] Section 5.1 details all 12 models, 9 benchmark tasks, 3+2 conditions (direct, short CoT, budget CoT, plus tool use and budget sweep), difficulty levels, instance counts, and statistical methodology (bootstrap CIs with 10,000 resamples).
7. **Experiment Statistical Significance.** Does the paper report error bars suitably and correctly? [Yes] All figures include 95% bootstrap confidence intervals. Tables report CI ranges. Statistical tests (McNemar's, ANOVA, Jonckheere-Terpstra) are used where appropriate with p-values reported.
8. **Experiments Compute Resources.** Does the paper report the computational resources used? [Yes] Section 5.1 reports the total evaluation cost (approximately $380 across API providers, dominated by frontier model evaluations) and instance counts. No model training was performed.
9. **Code Of Ethics.** Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics? [Yes] This work evaluates existing models on synthetic benchmarks. No human subjects, no harmful applications, no private data.
10. **Broader Impacts.** Does the paper discuss both potential positive societal impacts and negative societal impacts of the work? [Yes] Section 6 discusses practical implications for responsible AI deployment (matching interventions to failure types) and notes that the taxonomy could be misused to target model weaknesses.
11. **Safeguards.** Does the paper describe safeguards that have been put in place for responsible disclosure of data or code? [NA] The benchmarks are synthetic mathematical/logical tasks. No safety-critical data or capabilities are involved.
12. **Licenses.** Are the creators or original owners of assets (e.g., code, data, models) used in the paper properly credited and are the license and terms of use explicitly mentioned and properly respected? [Yes] All models are accessed via official APIs or open-source licenses. Model names and versions are fully specified.
13. **New Assets.** Are new assets introduced in the paper well documented and is the documentation provided alongside the assets? [Yes] The ReasonGap benchmark suite is fully documented in Appendix B with generation algorithms, difficulty parameters, and ground truth computation.
14. **Crowdsourcing and Research with Human Subjects.** For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots? [NA] No crowdsourcing or human subjects research.
15. **IRB Approvals or Equivalent.** Does the paper describe potential risks incurred by study participants? [NA] No human participants.
16. **Declaration of LLM usage.** Does the paper describe LLM usage in core methods, if important, original, or non-standard? [NA] LLMs are the subject of evaluation, not used as a methodological tool. Standard editing assistance was used for writing but does not impact core methodology or scientific rigor.